PipeSense-ARM: Safe Runtime Pipeline Adaptation in an Educational Embedded Core

Tyler Lee
Electrical and Computer Engineering
NYU Tandon School of Engineering
Brooklyn, NY 11201, USA
tl2966@nyu.edu

***Abstract*—Static pipeline policies cannot match every phase of an embedded workload. PipeSense-ARM is a SystemVerilog research artifact that places a rolling-window event observer, a hysteretic mode controller, and a drain-before-switch reconfiguration unit around a five-stage ARM-like educational core. The controller selects among normal, branch, memory, hazard, and low-activity modes without benchmark labels or software intervention. Across 13 embedded-style workloads and six configurations, adaptive execution reduces total cycles by 7.59% and an activity-energy proxy by 5.78% relative to static normal mode; nine workloads improve, three tie, and one regresses. The adaptive policy nevertheless remains an average 8.90% behind the hindsight best fixed mode, exposing policy headroom. All 78 directed runs match a sequential ISA reference and report zero safety faults or timeouts. A 500-seed constrained-random regression produces 2,500 mode-result rows with zero assertion failures, safety faults, or timeouts. Three bounded formal jobs also pass. Yosys reports a 29.49% integrated generic-cell delta for the production adaptive RTL against a common core shell. The result is a reproducible control artifact with explicit performance, safety, and cost limits rather than an ARM-compatible processor or calibrated hardware implementation.**

***Index Terms*—adaptive microarchitecture, embedded processors, hardware monitoring, pipeline reconfiguration, SystemVerilog**

I. Introduction

Embedded programs can move between control-heavy loops, streaming memory accesses, dependency chains, and low-utilization intervals. Program phases have long motivated sampling and reconfigurable hardware [1], [2]. A fixed pipeline policy is easy to verify, but it can retain a branch penalty during a branch-heavy interval or pay memory waits when a local mitigation would be more useful. Software and firmware can react to coarse counters, but a small in-core controller can observe the causes of stalls directly and respond without an interrupt or runtime service.

This paper studies a narrow question: can a hardware-resident feedback loop classify pipeline behavior, select a matching microarchitectural mode, and change modes without corrupting in-flight execution? PipeSense-ARM implements that loop in a five-stage IF/ID/EX/MEM/WB processor model. The observer counts branches, memory waits, load-use hazards, stalls, valid stages, and retirement over short windows. A controller applies stability and residency conditions, and a separate reconfiguration unit gates fetch and drains the pipeline before committing a requested mode.

The contribution is the integration and evaluation of this inspectable loop, not a claim that its individual mechanisms are new. The artifact contributes: 1) an event-driven phase observer and hysteretic controller attached to a sequential RTL core; 2) an explicit drain-before-switch contract checked by simulation assertions, architectural reference comparison, and randomized stress; and 3) an evaluation that preserves unfavorable results through fixed baselines, a hindsight oracle, parameter sweeps, ablations, and synthesis cost proxies. This combination exposes both the benefit and the present cost of hardware-native adaptation.

II. Background and Related Work

Branch prediction and memory buffering are established ways to reduce control and memory penalties [3], [4]. Reconfigurable memory hierarchies further show that a hardware structure can trade performance and energy as behavior changes [5]. Phase-characterization work uses recurring program signatures to select representative behavior or manage multiple configurations [1], [2]. PipeSense-ARM uses the same broad observation that behavior changes over time, but it does not construct basic-block or working-set signatures. It classifies direct pipeline events with threshold logic over a short window.

Architectural simulators and activity models support early design exploration [6], [7]. Their results still depend on the realism of the modeled machine and workload. Accordingly, the energy value reported here is a sum of active pipeline slots with a fixed activity term outside low-activity mode; it is not power or energy measured from hardware. Likewise, the branch and memory modes are experimental mechanisms rather than a production predictor or cache.

Runtime reconfiguration adds a correctness obligation beyond performance. SystemVerilog assertions provide clocked checks close to RTL behavior [8], [9]. PipeSense-ARM combines such checks with instruction tags and a sequential ISA model. The novelty boundary is therefore modest: a compact artifact connects direct phase observation, hysteretic control, conservative switching, and reproducible negative as well as positive evidence in one educational pipeline.

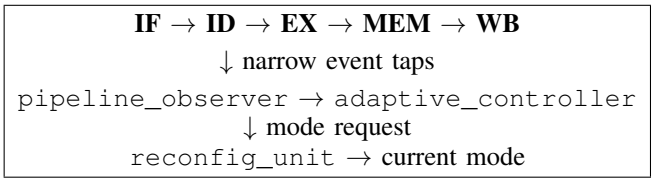

Fig. 1. Closed-loop organization of PipeSense-ARM. The policy observes only pipeline events; mode changes are committed by a separate safety unit.

III. Design and Methodology

A. Core and Adaptive Modes

The modeled core supports 32-bit educational encodings for arithmetic and logical operations, load, store, compare, conditional branch, no-operation, and a simulation-only halt sentinel. It includes valid bits, forwarding, load-use interlocking, branch flushes, deterministic memory waits, writeback, and performance counters. It is ARM-like in organization and instruction style but is not ARM compatible.

Five modes expose distinct timing or activity behavior. Normal mode uses the default branch, memory, and hazard paths. Branch mode resolves eligible branches in decode and reduces the modeled flush from two slots to one. Memory mode mitigates periodic data-memory wait requests, representing a small buffer or prefetch effect. Hazard mode enables an additional load-result bypass and suppresses the corresponding load-use stall. Low-activity mode removes a fixed term from the activity proxy. These modes make controller decisions observable while keeping the prototype small; they should not be interpreted as complete commercial mechanisms.

B. Observer and Controller

Figure 1 shows the feedback path. The observer samples stage occupancy, branch-taken and flush events, load-use hazards, memory waits, front-end stalls, cycles, and retired instructions. At the end of each parameterized observation window, counters are compared with thresholds for branch-heavy, memory-stall, load-use-hazard, front-end-stall, and idle/low-utilization behavior. Otherwise, the window is labeled balanced. The observer resets its counters after classification, so decisions depend on recent events rather than a cumulative program profile.

The controller maps branch-heavy and front-end-stall phases to branch mode, memory-stall phases to memory mode, load-use phases to hazard mode, and idle/low-utilization phases to low-activity mode. Balanced behavior is sticky: it retains the current mode instead of forcing a switch to normal. A stable phase counter prevents one anomalous window from immediately changing policy. Mode-specific minimum residency requirements provide additional hysteresis; memory mode can be entered sooner than modes intended for less dominant events. The observer window, event thresholds, and minimum residency are parameters used in the sensitivity study.

C. Safe Reconfiguration

A controller request does not directly update the visible mode. The reconfiguration unit first stops new fetches. Existing instructions continue through IF/ID, ID/EX, EX/MEM, and MEM/WB until all four pipeline registers are invalid and no memory wait is active. The unit then commits the latched mode, acknowledges completion, and records both the event and its stall cycles. The safe-boundary predicate is

$$safe = pipeline\_empty \wedge \neg mem\_wait. \tag{1}$$

This conservative protocol can cost more than selective quiescence, but it prevents an instruction from crossing a mode change that alters branch, forwarding, or memory timing.

Simulation monitors check that retirement tags increase monotonically, no two valid stages contain the same dynamic tag, mode changes coincide with the safe boundary and completion acknowledgement, fetch remains gated during a drain, the mode remains stable while reconfiguration is active, and the stall remains bounded. The HDL result for each workload and mode is also compared with a sequential ISA model using retired count and final architectural-state hash. Bounded SymbiYosys jobs check the production reconfiguration unit to depth 24, abstract token conservation to depth 9, and abstract no-double-commit behavior to depth 14. No full-core formal proof is claimed.

IV. Evaluation

The directed matrix contains 13 workloads: arithmetic-heavy, branch-heavy, memory-heavy, load-use-heavy, mixed-control, tiny-FIR, Dhrystone-style and CoreMark-style toy ports, generated-style FIR and PID loops, long-FIR stress, PID-phase stress, and randomized-memory-latency stress. The original microbenchmarks isolate specific causes; the longer programs reduce dependence on tiny loops without pretending to be full benchmark-suite ports. Every workload runs in static normal, fixed branch, fixed memory, fixed hazard, fixed low-activity, and adaptive configurations, producing 78 directed rows.

Metrics are cycles, retired instructions, instructions per cycle (IPC), stall and flush cycles, memory waits, load-use stalls, reconfiguration events and penalty, activity-energy proxy, safety faults, timeout status, and architectural hash. Adaptive mode is compared with static normal and with the best fixed mode chosen after observing each workload. The latter is not a deployable policy; it is a hindsight oracle that reveals remaining opportunity.

The parameter study crosses observation windows of 16, 32, and 64 cycles; minimum residencies of 8, 24, and 48 cycles; and tight, medium, and loose threshold profiles. The resulting 27 configurations each rerun all 78 rows, for 2,106 simulations. Ablations disable the observer, disable the controller, or analytically remove measured reconfiguration penalty. A separate generator runs legal constrained-random programs for 500 seeds and five execution modes. The Docker evaluation uses Icarus Verilog 12.0, Yosys 0.33, SymbiYosys, Z3, and CVC4. Scripts validate row coverage, timeouts, safety fields, reference-model equality, sweep output, bounded formal results, and the public result summary. Source

TABLE I
ADAPTIVE CYCLE RESULTS. ORACLE GAP IS RELATIVE TO THE BEST FIXED MODE; THE FINAL ORACLE ENTRY IS THE MEAN OF PER-WORKLOAD GAPS.

Workload	Normal	Adapt.	Red. (%)	Oracle (%)
Arithmetic	30	30	0.00	-3.45
Branch	128	114	10.94	-8.57
CoreMark-toy	162	162	0.00	-8.00
Dhrystone-toy	133	133	0.00	-7.26
DSP-FIR	192	194	-1.04	-15.48
Load-use	205	178	13.17	-5.33
Long-FIR	876	792	9.59	-1.80
Memory	231	162	29.87	-10.20
Mixed-control	131	127	3.05	-20.95
PID-codegen	192	188	2.08	-5.62
PID-phase	653	620	5.05	-1.47
Random-memory	215	204	5.12	-7.94
Tiny-FIR	159	152	4.40	-19.69
Total / mean	3307	3056	7.59	-8.90

and commands are available at https://github.com/Tylerlee102/PipeSense-ARM.

V. RESULTS

A. Performance and Oracle Comparison

All 78 directed runs terminate, report zero safety faults, and pass the retired-count and architectural-hash comparison. Table I reports the complete cycle result. Adaptive execution uses 3,056 cycles versus 3,307 for static normal, a 7.59% reduction. Average per-workload cycle reduction is 6.33%, average IPC improvement is 7.71%, and the total activity-energy proxy falls from 19,162 to 18,055, or 5.78%. Nine workloads improve, three tie, and generated DSP-FIR regresses by 1.04% because two mode changes cost eight cycles in a short run.

The oracle result limits the performance claim. Adaptive execution is behind the best fixed choice on every workload, with a mean reported gap of 8.90%. Long-FIR and PID-phase come within 1.80% and 1.47%, respectively, while mixed-control and tiny-FIR remain 20.95% and 19.69% behind. The controller therefore captures useful phase behavior but is not an optimal online policy. The large static-normal gain on memory-heavy (29.87%) coexists with a 10.20% oracle gap, illustrating why both baselines are necessary.

B. Sensitivity, Safety, and Cost

The full sweep contains 1,755 adaptive-versus-fixed comparison cells. Adaptive wins 478 and does not win 1,277. Against static normal alone, mean cycle reduction increases from 1.91% at a 16-cycle window to 3.72% at 32 cycles and 4.29% at 64 cycles, averaged across the other sweep dimensions. Tight, medium, and loose thresholds average 3.98%, 3.22%, and 2.72%, respectively. These results show policy sensitivity rather than a universally robust default.

Table II summarizes the remaining evidence. Disabling either the observer or controller returns adaptive execution to the 3,307-cycle static-normal aggregate, so the improvement is not caused by a hidden default mode. Removing all measured reconfiguration penalty would reduce cycles only from 3,056 to 3,009, a further 1.54%; phase selection is therefore a larger opportunity than drain latency alone.

TABLE II
ABLATION, SAFETY, AND GENERIC SYNTHESIS EVIDENCE.

Evidence	Result	Relative outcome
Observer disabled	3307 cycles	+8.21% cycles
Controller disabled	3307 cycles	+8.21% cycles
Zero-cost switching	3009 cycles	-1.54% cycles
Safety fuzz	2500 rows	0 failures/timeouts
Baseline core shell	1838 cells	reference
Adaptive RTL sum	550 cells	29.92% of shell
Integrated top	2380 cells	+29.49% cells

The 500-seed fuzz run reports zero assertion failures, safety faults, timeouts, or nonzero simulator exits across 2,500 mode-result rows. This is stress evidence, not exhaustive verification. Yosys maps the baseline shell to 1,838 generic cells and the integrated top to 2,380, a 29.49% delta. Standalone production observer, controller, and reconfiguration modules map to 319, 180, and 51 cells, respectively. Both runs use the same core shell, and no debug-only wide outputs force control state into the count. These figures are neither FPGA utilization nor ASIC area, but they show that adaptation remains a nontrivial cost beside a very small teaching core.

VI. DISCUSSION AND LIMITATIONS

The results support a scoped conclusion: direct pipeline events can drive a closed-loop RTL controller that improves over a static normal policy while preserving the modeled architectural and switching invariants. The negative oracle result and synthesis estimate are equally important. They show that the current classifier leaves substantial mode-selection headroom and that even bounded control logic is measurable beside a small teaching core.

Internal validity is limited by stylized mode effects. Memory optimization suppresses a deterministic wait request rather than modeling a cache or prefetch queue. Branch optimization changes resolution timing rather than implementing a predictor. The low-activity result is based on an activity sum, not voltage, capacitance, frequency, or measured power. Yosys generic cells do not establish timing closure or physical area. These proxies are suitable for testing the control loop, not for claiming production speed, energy, or cost.

External validity is limited by the educational ISA and workload suite. The toy and generated kernels are transparent and reference-checkable, but they do not replace compiler-generated Embench, CoreMark, or application traces with a real cache hierarchy, interrupts, and exceptions. Safety evidence is also bounded: simulation assertions and randomized programs cover observed traces, while the passing bounded formal jobs check only abstract or isolated properties. Full-core token conservation and liveness remain future proof obligations.

A stronger implementation should replace counter banks with shared or sampled counters, implement concrete branch

and memory structures, tune policy on training workloads separate from evaluation workloads, and measure area, timing, and power on a named FPGA or standard-cell target. The existing oracle, sweep, ablation, reference, and safety checks provide a basis for judging those changes without discarding unfavorable rows.

VII. Conclusion

PipeSense-ARM demonstrates a complete hardware-resident adaptation loop in a small sequential processor model. A rolling-window observer identifies local bottlenecks, a hysteretic controller requests a matching mode, and a separate unit commits changes only after a safe drain. The adaptive configuration reduces total cycles by 7.59% and the activity-energy proxy by 5.78% relative to static normal while passing directed reference checks and randomized safety stress. Its 8.90% mean gap to the hindsight best fixed mode and 29.49% generic-cell overhead prevent a broader efficiency claim. The artifact is therefore most useful as a reproducible foundation for safer and more cost-effective adaptive embedded microarchitectures.

References

[1] T. Sherwood, E. Perelman, G. Hamerly, and B. Calder, "Automatically characterizing large scale program behavior," in *Proceedings of the 10th International Conference on Architectural Support for Programming Languages and Operating Systems*, 2002, pp. 45–57.

[2] A. S. Dhodapkar and J. E. Smith, "Managing multi-configuration hardware via dynamic working set analysis," in *Proceedings of the 29th Annual International Symposium on Computer Architecture*, 2002, pp. 233–244.

[3] J. E. Smith, "A study of branch prediction strategies," in *Proceedings of the 8th Annual Symposium on Computer Architecture*, 1981, pp. 135–148.

[4] N. P. Jouppi, "Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers," in *Proceedings of the 17th Annual International Symposium on Computer Architecture*, 1990, pp. 364–373.

[5] R. Balasubramonian, D. H. Albonesi, A. Buyuktosunoglu, and S. Dwarkadas, "Memory hierarchy reconfiguration for energy and performance in general-purpose processor architectures," in *Proceedings of the 33rd Annual IEEE/ACM International Symposium on Microarchitecture*, 2000, pp. 245–257.

[6] T. Austin, E. Larson, and D. Ernst, "Simplescalar: An infrastructure for computer system modeling," *Computer*, vol. 35, no. 2, pp. 59–67, 2002.

[7] D. Brooks, V. Tiwari, and M. Martonosi, "Wattch: A framework for architectural-level power analysis and optimizations," in *Proceedings of the 27th Annual International Symposium on Computer Architecture*, 2000, pp. 83–94.

[8] "IEEE Standard for SystemVerilog–Unified Hardware Design, Specification, and Verification Language," IEEE, 2023.

[9] H. D. Foster, A. C. Krolnik, and D. J. Lacey, *Assertion-Based Design*. Kluwer Academic Publishers, 2004.